

LC 1138: Alphabet Board Path

Tags: Zulily, Google

© Authored by Arpit Raj, LNMIIT Jaipur | July 31, 2026

Problem Overview

Given a board (typing board) and a (string to type), we need to return the movement required to type that word.

a	b	c	d	e
f	g	h	i	j
k	l	m	n	o
p	q	r	s	t
u	v	w	x	y
z				

Example: Then to type **aadya**

We need to return this: **!! RRR! DDDDR! UUUULLLL!**

Proposed Approaches

Approach 1: Breadth-First Search (BFS)

Run BFS from current character to next character and store path.

Time Complexity (TC): $O(26 \times \text{target length})$ → valid for any word (since there are only 26 cells).

Approach 2: Coordinate Math

The board is fixed, hence the cells are fixed.

• a = (0, 0)

- $b = (0, 1)$
- ...
- $z = (5, 0)$

Example 1: $\text{curr} = a = (0, 0)$ and $\text{next} = d = (0, 3)$

Hence 3 times R: **RRR**

Example 2: $(3, 2) \rightarrow (0, 1)$

So: **UUUL**

Note: z is alone. Going to z means horizontal move first then go down.

C++ Implementation

Below is the BFS implementation code, formatted and preserved exactly as written:

```

class Solution {
public:
    string bfs(int sr, int sc, int er, int ec, vector<string>& board) {
        queue<pair<pair<int, int>, string>> q;
        // queue me r, c, path ab tak ka

        vector<vector<int>> seen(6, vector<int>(5, 0));
        q.push({{sr, sc}, ""});
        seen[sr][sc] = 1;

        vector<pair<pair<int, int>, char>> dir = {{{-1, 0}, 'U'}, {{0, -1}, 'L'}, {{1, 0}, 'D'}};

        while (!q.empty()) {
            int r = q.front().first.first;
            int c = q.front().first.second;
            string path = q.front().second;
            q.pop();

            if (r == er && c == ec) return path;
            for (auto &it : dir) {
                int nr = r + it.first.first;
                int nc = c + it.first.second;

                if (nr >= 0 && nc >= 0 && nr < 6 && nc < 5) {
                    if (!seen[nr][nc] && board[nr][nc] != '?') {
                        seen[nr][nc] = 1;
                        q.push({{nr, nc}, path + it.second});
                    }
                }
            }
        }

        return "-1";
    }

    string alphabetBoardPath(string target) {
        vector<string> board = {"abcde", "fghij", "klmno", "pqrst", "uvwxy", "z????"};

        string ans = "";
        vector<pair<int, int>> pos(26);
        for (int i = 0; i < 6; i++) {
            for (int j = 0; j < 5; j++) {
                if (board[i][j] == '?') continue;
                pos[board[i][j] - 'a'] = {i, j};
            }
        }

        int cr = 0;
        int cc = 0;
        for (char ch : target) {
            int nr = pos[ch - 'a'].first;
            int nc = pos[ch - 'a'].second;

            ans += bfs(cr, cc, nr, nc, board);
            ans += "!";
            cr = nr;
        }
    }
};

```

```
        cc = nc;  
    }  
  
    return ans;  
}  
};
```